

DOKUMENTASI TEKNIS

VerbaClassify

Sistem Klasifikasi Respon Masyarakat Terhadap
Isu Pelecehan Seksual Secara Verbal pada Platform X
Menggunakan Algoritma ID3 Modifikasi

Penulis	Rizka Mardiah Putri Buyung Lubis
NIM	220170183
Program Studi	Teknik Informatika
Institusi	Universitas Malikussaleh
Tahun	2026
Versi Dokumen	1.0

Flask • Python • Supabase • ID3 Modifikasi • TF-IDF • SMOTE • NLTK • PySastrawi

■ DAFTAR ISI

1. **Gambaran Umum Sistem**
2. **Struktur Folder dan File**
3. **Konfigurasi dan Environment**
4. **Skema Database (schema.sql)**
5. **File: app.py — Entry Point & Routing**
 - 5.1. Konfigurasi Aplikasi
 - 5.2. Dekorator Autentikasi
 - 5.3. Public Routes
 - 5.4. Auth Routes
 - 5.5. Admin Routes
 - 5.6. API Endpoints
6. **File: database.py — Data Access Layer**
 - 6.1. Fungsi Koneksi
 - 6.2. Fungsi Dataset
 - 6.3. Fungsi Raw Data
 - 6.4. Fungsi Experiments
 - 6.5. Fungsi Depth Results
 - 6.6. Fungsi Predictions
 - 6.7. Fungsi Statistik
7. **File: preprocessing.py — Pipeline NLP**
 - 7.1. Inisialisasi Library
 - 7.2. Kamus Stopwords dan Slang
 - 7.3. Fungsi Preprocessing Per Tahap
 - 7.4. Pipeline Utama
8. **File: pipeline.py — Machine Learning Pipeline**
 - 8.1. Fungsi run_experiment()
 - 8.2. Fungsi run_depth_sweep()
 - 8.3. Manajemen Model
 - 8.4. Fungsi predict_single()
9. **File: id3_modified.py — Algoritma ID3 Modifikasi**
 - 9.1. Formula Matematika
 - 9.2. Fungsi Entropy dan Information Gain
 - 9.3. Kelas ID3ModifiedNode
 - 9.4. Kelas ID3ModifiedClassifier
10. **Template HTML (Folder templates/)**

11. File Models & Uploads
12. Alur Kerja Sistem (End-to-End)
13. API Reference
14. Panduan Setup & Instalasi
15. Dependensi (requirements.txt)

■ 1. Gambaran Umum Sistem

VerbaClassify adalah aplikasi web berbasis Flask yang dirancang sebagai platform penelitian skripsi untuk mengklasifikasikan respon masyarakat terhadap isu pelecehan seksual secara verbal di platform X (Twitter). Sistem ini mengimplementasikan algoritma ID3 Modifikasi — sebuah pengembangan dari algoritma pohon keputusan klasik ID3 yang menggunakan formula entropy berbasis logaritma natural — sebagai metode klasifikasi utama.

Sistem memiliki dua mode akses utama:

Mode Akses	Pengguna	Kemampuan
Publik (tanpa login)	Masyarakat umum	Klasifikasi teks komentar, lihat info model terbaik, akses API klasifikasi
Admin (dengan login)	Peneliti / Administrator	Upload dataset CSV, konfigurasi & jalankan eksperimen training, analisis hasil, kelola model

Arsitektur Sistem

VerbaClassify menggunakan arsitektur tiga lapis (three-tier):

Lapisan	Komponen	Teknologi
Presentation Layer	Template HTML (14 halaman)	HTML/CSS vanilla, Jinja2
Business Logic Layer	Flask Routes, Pipeline ML	Python 3.x, Flask, scikit-learn
Data Layer	Database cloud	Supabase (PostgreSQL), File .pkl di disk

2. Struktur Folder dan File

Berikut adalah struktur lengkap folder dan file dalam proyek VerbaClassify beserta fungsinya:

Path File/Folder	Jenis	Fungsi Utama
skripsi_app/	Direktori Root	Folder utama aplikasi, berisi semua file proyek
skripsi_app/app.py	Python — Entry Point	File utama Flask: routing, autentikasi, background task, REST API
skripsi_app/database.py	Python — DAL	Data Access Layer: semua operasi CRUD ke Supabase
skripsi_app/preprocessing.py	Python — NLP	Pipeline preprocessing teks Bahasa Indonesia (6 tahap)
skripsi_app/pipeline.py	Python — ML	Pipeline machine learning: training, evaluasi, simpan/muat model, prediksi
skripsi_app/id3_modified.py	Python — Algoritma	Implementasi algoritma ID3 Modifikasi dengan formula entropy ln
skripsi_app/schema.sql	SQL	Definisi schema 7 tabel PostgreSQL untuk Supabase
skripsi_app/requirements.txt	Config	Daftar dependensi Python yang perlu diinstal
skripsi_app/.env	Config (Rahasia)	Variabel lingkungan: URL Supabase, API key, kredensial admin
skripsi_app/uploads/	Direktori	Penyimpanan sementara file CSV yang diunggah admin
skripsi_app/models/	Direktori	Penyimpanan file model terlatih (model_.pkl)
skripsi_app/templates/	Direktori	Semua file template HTML Jinja2 (14 file)
skripsi_app/__pycache__/	Direktori	Cache bytecode Python (dibuat otomatis, abaikan)

■ ■ 3. Konfigurasi dan Environment

Konfigurasi sistem menggunakan file **.env** yang dibaca oleh library python-dotenv. File ini TIDAK boleh di-commit ke version control karena berisi kredensial sensitif.

Variabel	Default	Keterangan
SUPABASE_URL	(wajib diisi)	URL project Supabase, format: <code>https://xxx.supabase.co</code>
SUPABASE_KEY	(wajib diisi)	API key Supabase (anon key atau service_role key)
SECRET_KEY	rizka_skripsi_2026	Secret key Flask untuk enkripsi session cookie
ADMIN_USERNAME	admin	Username untuk login ke panel admin
ADMIN_PASSWORD	rizka2026	Password untuk login ke panel admin (ubah di production!)

Contoh isi file .env:

```
SUPABASE_URL=https://lbzrvjrcyzmszgkijmoq.supabase.co
SUPABASE_KEY=sb_publishable_XXXXXXXXXXXXXXXXXXXX
SECRET_KEY=rizka_skripsi_secret_key_2026
ADMIN_USERNAME=admin
ADMIN_PASSWORD=rizka2026
```

■ ■ 4. Skema Database (schema.sql)

Sistem menggunakan Supabase (PostgreSQL) sebagai database cloud. Terdapat 7 tabel yang saling berelasi melalui foreign key constraints dengan cascade delete.

Tabel: datasets

Kolom	Tipe	Keterangan
id	BIGSERIAL PK	ID unik dataset (auto-increment)
filename	TEXT	Nama file CSV yang diunggah
total_data	INTEGER	Jumlah total baris data valid
positif_count	INTEGER	Jumlah data berlabel positif
negatif_count	INTEGER	Jumlah data berlabel negatif
uploaded_at	TIMESTAMPTZ	Waktu upload (default: NOW())
notes	TEXT	Catatan opsional dari admin

Tabel: raw_data

Kolom	Tipe	Keterangan
id	BIGSERIAL PK	ID unik baris data
dataset_id	BIGINT FK → datasets	Relasi ke tabel datasets (CASCADE DELETE)
full_text	TEXT NOT NULL	Teks komentar asli dari platform X
label	TEXT CHECK	Label kelas: "positif" atau "negatif"
preprocessed_text	TEXT	Hasil preprocessing teks (dihitung saat upload)
created_at	TIMESTAMPTZ	Waktu insert

Tabel: experiments

Kolom	Tipe	Keterangan
id	BIGSERIAL PK	ID unik eksperimen
dataset_id	BIGINT FK → datasets	Dataset yang digunakan
proportion_label	TEXT	Label proporsi: "80:20", "70:30"
train_ratio / test_ratio	FLOAT	Rasio pembagian data (0.0–1.0)
max_depth	INTEGER	Parameter kedalaman pohon ID3 (default: 3)
use_smote	BOOLEAN	Apakah SMOTE diaktifkan
accuracy, precision_score, recall_score, f1_score	FLOAT	Metrik evaluasi keseluruhan
tp, tn, fp, fn	INTEGER	Komponen confusion matrix

execution_time, train_time, predict_time	FLOAT	Waktu eksekusi dalam detik
is_best	BOOLEAN	Apakah ini model terbaik yang aktif
status	TEXT CHECK	Status: pending → running → done / error
error_msg	TEXT	Pesan error jika status=error

Tabel: experiment_details

Kolom	Tipe	Keterangan
id	BIGSERIAL PK	ID unik detail
experiment_id	BIGINT FK → experiments	Relasi ke eksperimen (CASCADE DELETE)
class_label	TEXT	Nama kelas: "positif" atau "negatif"
precision_val, recall_val, f1_val	FLOAT	Metrik per kelas
support	INTEGER	Jumlah sampel aktual kelas ini

Tabel: depth_results

Kolom	Tipe	Keterangan
id	BIGSERIAL PK	ID unik hasil depth sweep
experiment_id	BIGINT FK → experiments	Relasi ke eksperimen
max_depth	INTEGER	Nilai depth yang diuji
accuracy, precision_score, recall_score, f1_score	FLOAT	Metrik evaluasi pada depth ini
execution_time	FLOAT	Waktu eksekusi pada depth ini

Tabel: predictions

Kolom	Tipe	Keterangan
id	BIGSERIAL PK	ID unik prediksi
input_text	TEXT	Teks asli yang diinput pengguna (maks 1000 char)
preprocessed_text	TEXT	Hasil preprocessing teks input
predicted_label	TEXT CHECK	Hasil prediksi: "positif" atau "negatif"
confidence	FLOAT	Nilai kepercayaan prediksi (0.0–1.0)
experiment_id	BIGINT FK → experiments	Model yang digunakan untuk prediksi
created_at	TIMESTAMPTZ	Waktu prediksi

■ 5. File: app.py — Entry Point & Routing

File **app.py** adalah entry point utama aplikasi Flask. File ini mendefinisikan semua URL routing, logika autentikasi admin, mekanisme background task untuk training model, dan REST API endpoint. Semua modul lain (database, pipeline, preprocessing) diimpor secara lazy (di dalam fungsi) untuk efisiensi memori.

5.1 Konfigurasi Aplikasi

Konfigurasi	Nilai / Sumber	Keterangan
app.secret_key	env: SECRET_KEY	Secret key Flask untuk enkripsi cookie session
MAX_CONTENT_LENGTH	50 MB	Batas ukuran file upload maksimum
UPLOAD_FOLDER	skripsi_app/uploads/	Direktori penyimpanan file CSV yang diunggah
ADMIN_USERNAME	env: ADMIN_USERNAME	Username admin panel
ADMIN_PASSWORD	env: ADMIN_PASSWORD	Password admin panel
_task_status (dict)	In-memory	Dictionary tracking status background training task

5.2 Dekorator Autentikasi

login_required(f)

Dekorator Flask yang melindungi route admin dari akses tanpa autentikasi. Mengecek keberadaan key "admin_logged_in" di Flask session. Jika tidak ada, redirect ke halaman login dengan pesan flash warning. Digunakan dengan sintaks `@login_required` di atas definisi fungsi route yang ingin dilindungi.

Parameter:

f — Fungsi view Flask yang akan dilindungi

Return:

Fungsi wrapper yang menambahkan pengecekan autentikasi sebelum memanggil fungsi asli

5.3 Public Routes (Tanpa Login)

Route	Method	Fungsi	Keterangan
/	GET	index()	Halaman beranda — menampilkan info model terbaik dan statistik prediksi
/classify	GET, POST	classify()	Form klasifikasi teks publik — validasi input, prediksi, simpan hasil ke DB
/about	GET	about()	Halaman tentang sistem — penjelasan algoritma dan metodologi

Alur kerja classify() (POST):

1. Cek ketersediaan model terbaik (`get_best_experiment()`)

- Validasi input: tidak kosong, minimal 5 karakter
- Panggil `predict_single(text, best_experiment_id)`
- Jika berhasil: simpan ke DB dengan `insert_prediction()`
- Render hasil (label, confidence) ke template `classify.html`

5.4 Auth Routes

Route	Method	Fungsi	Keterangan
/admin/login	GET, POST	login()	Form login admin — validasi username/password vs environment variable
/admin/logout	GET	logout()	Hapus seluruh session (<code>session.clear()</code>), redirect ke beranda

5.5 Admin Routes (Perlu Login)

Route	Method	Fungsi	Keterangan
/admin	GET	dashboard()	Dashboard utama: statistik ringkasan sistem, list dataset & eksperimen terbaru
/admin/datasets	GET	datasets()	Daftar semua dataset yang diunggah, diurutkan dari terbaru
/admin/datasets/upload	GET, POST	upload_dataset()	Upload CSV baru: validasi format, preprocessing batch, insert ke DB
/admin/datasets/	GET	dataset_detail()	Detail dataset: preview data dengan paginasi 20 baris/halaman
/admin/datasets//delete	POST	delete_dataset_route()	Hapus dataset beserta semua data terkait (cascade)
/admin/experiments	GET	experiments_list()	Daftar semua eksperimen selesai, tandai mana yang <code>is_best</code>
/admin/experiments/run	GET, POST	run_experiment_route()	Form konfigurasi & mulai training di background thread
/admin/experiments/progress/	GET	experiment_progress()	Halaman monitoring progress dengan JavaScript polling
/admin/experiments/	GET	experiment_detail()	Detail hasil eksperimen: metrik, grafik depth sweep, confusion matrix
/admin/experiments//set-best	POST	set_best_route()	Tandai eksperimen sebagai model terbaik (<code>is_best=True</code>)

/admin/statistics	GET	statistics()	Statistik komprehensif: perbandingan eksperimen, distribusi prediksi
/admin/predictions	GET	predictions_list()	Riwayat 100 prediksi terbaru dari pengguna

Alur upload_dataset() (POST) — detail:

1. Terima file dari form (field "file"), validasi ekstensi .csv
2. Sanitasi nama file dengan secure_filename() dari Werkzeug
3. Coba baca CSV dengan encoding: utf-8 → utf-8-sig → latin-1 → cp1252
4. Normalisasi nama kolom (lowercase, hapus BOM \uffff)
5. Validasi kolom wajib: "full_text" dan "label"
6. Filter baris: hanya label positif/negatif, teks > 2 karakter
7. Jalankan preprocess_batch() untuk semua teks sekaligus
8. Insert metadata ke tabel datasets
9. Insert semua baris ke raw_data dalam batch 500 record

Mekanisme Background Task (run_experiment_route):

Proses training dijalankan dalam threading.Thread(daemon=True) agar web server tidak blocking. Status setiap task disimpan dalam dictionary _task_status (in-memory) dengan format:

```
_task_status[task_id] = {
    "status" : "running" | "done" | "error" | "not_found",
    "progress": int (0-100),
    "message" : str (pesan status terakhir)
}
```

Frontend melakukan polling ke /api/task-status/<task_id> setiap beberapa detik untuk update UI.

5.6 API Endpoints

Endpoint	Method	Auth	Keterangan
/api/classify	POST	Tidak	Klasifikasi teks via JSON — request: {"text": "..."}, response: {label, confidence, preprocessed}
/api/stats	GET	Tidak	Statistik ringkasan sistem (total dataset, eksperimen, prediksi, akurasi terbaik)
/api/task-status/	GET	Admin	Status background task training (untuk polling progress)
/api/experiments//depth-results	GET	Tidak	Data hasil sweep max_depth untuk grafik Chart.js
/api/experiments//model-detail	GET	Admin	Detail model pickle: vocab size, top-20 feature importance

■ 6. File: database.py — Data Access Layer

File **database.py** berfungsi sebagai Data Access Layer (DAL) yang memisahkan logika bisnis dari detail koneksi database. Semua operasi CRUD ke Supabase harus melewati fungsi-fungsi di file ini. Menggunakan pola Singleton untuk instance Supabase client.

6.1 Fungsi Koneksi

`get_db()` → `Client`

Mengembalikan instance Supabase Client menggunakan pola Singleton. Instance hanya dibuat satu kali selama siklus hidup aplikasi. Membaca `SUPABASE_URL` dan `SUPABASE_KEY` dari environment.

Return:

`supabase.Client` — instance siap pakai untuk operasi database

6.2 Fungsi Dataset

Fungsi	Parameter	Return	Kegunaan
<code>insert_dataset()</code>	<code>filename</code> , <code>total</code> , <code>positif</code> , <code>negatif</code> , <code>notes</code>	<code>dict</code> <code>None</code>	Insert metadata dataset baru ke tabel <code>datasets</code>
<code>get_all_datasets()</code>	—	<code>list[dict]</code>	Ambil semua dataset,urut dari terbaru (<code>uploaded_at DESC</code>)
<code>get_dataset_by_id()</code>	<code>dataset_id: int</code>	<code>dict</code> <code>None</code>	Ambil satu dataset berdasarkan ID
<code>delete_dataset()</code>	<code>dataset_id: int</code>	<code>None</code>	Hapus dataset dan semua data terkait (<code>cascade</code>)

6.3 Fungsi Raw Data

Fungsi	Parameter	Return	Kegunaan
<code>insert_raw_data_batch()</code>	<code>records: list[dict]</code>	<code>None</code>	Batch insert data mentah (500 record/request) ke tabel <code>raw_data</code>
<code>get_raw_data_by_dataset()</code>	<code>dataset_id</code> , <code>limit=100</code> , <code>offset=0</code>	<code>list[dict]</code>	Ambil data mentah dengan paginasi untuk preview
<code>get_all_raw_texts_labels()</code>	<code>dataset_id: int</code>	<code>list[dict]</code>	Ambil SEMUA teks+label+preprocessed untuk training (stream 1000/batch)
<code>update_preprocessed_text()</code>	<code>row_id</code> , <code>preprocessed</code>	<code>None</code>	Update <code>preprocessed_text</code> satu baris tertentu

6.4 Fungsi Experiments

Fungsi	Keterangan
<code>insert_experiment()</code>	Buat record eksperimen baru dengan status "pending"
<code>update_experiment_result()</code>	Simpan semua metrik hasil training, ubah status → "done"
<code>update_experiment_error()</code>	Tandai eksperimen gagal, simpan pesan error, ubah status → "error"
<code>update_experiment_running()</code>	Ubah status eksperimen → "running"
<code>get_experiments_by_dataset()</code>	Ambil semua eksperimen yang menggunakan dataset tertentu
<code>get_experiment_by_id()</code>	Ambil satu eksperimen berdasarkan ID
<code>get_best_experiment()</code>	Ambil eksperimen terbaik (<code>is_best=True</code>) atau fallback ke akurasi tertinggi
<code>set_best_experiment()</code>	Set <code>is_best=True</code> satu eksperimen, reset semua yang lain ke False
<code>get_all_done_experiments()</code>	Ambil semua eksperimen berstatus "done",urut akurasi DESC
<code>insert_experiment_details()</code>	Simpan metrik per kelas (precision, recall, F1, support)
<code>get_experiment_details()</code>	Ambil metrik per kelas untuk satu eksperimen

6.5 Fungsi Depth Results

Fungsi	Keterangan
<code>insert_depth_results()</code>	Simpan hasil akurasi untuk setiap nilai <code>max_depth</code> dari sweep (depth 1-20)
<code>get_depth_results()</code>	Ambil hasil sweep depth untuk satu eksperimen, urut <code>max_depth</code> ASC

6.6 Fungsi Predictions

Fungsi	Keterangan
<code>insert_prediction()</code>	Simpan satu hasil prediksi pengguna ke tabel predictions
<code>get_recent_predictions()</code>	Ambil N prediksi terbaru, urut <code>created_at</code> DESC
<code>get_prediction_stats()</code>	Hitung distribusi label: {total, positif, negatif}

6.7 Fungsi Statistik

`get_dashboard_stats()` → dict

Mengumpulkan semua statistik ringkasan dalam satu panggilan tunggal untuk kebutuhan halaman dashboard. Memanggil beberapa fungsi database secara berurutan dan menggabungkan hasilnya.

Return:

{total_datasets, total_experiments, total_predictions, best_accuracy, best_experiment, pred_positif, pred_negatif}

■ 7. File: preprocessing.py — Pipeline NLP

File **preprocessing.py** mengimplementasikan pipeline preprocessing teks Bahasa Indonesia yang dirancang khusus untuk klasifikasi komentar media sosial. Pipeline terdiri dari 6 tahap yang dijalankan secara berurutan sesuai metodologi dalam proposal skripsi Rizka (2026).

7.1 Inisialisasi Library

Modul menggunakan strategi import dengan fallback: jika PySastrawi tidak terinstal, sistem tetap berjalan tanpa stemming. NLTK data (punct, punct_tab, stopwords) didownload otomatis jika belum tersedia.

Library	Fungsi dalam Preprocessing
PySastrawi (StemmerFactory)	Stemming bahasa Indonesia menggunakan algoritma Nazief-Adriani
PySastrawi (StopWordRemoverFactory)	Daftar stopwords baku bahasa Indonesia
NLTK (word_tokenize)	Tokenisasi teks dengan mode Indonesian
re (regex)	Pembersihan noise menggunakan ekspresi reguler

7.2 Kamus Stopwords dan Slang

Modul mendefinisikan dua resource kunci yang dikombinasikan untuk mengoptimalkan preprocessing komentar media sosial Bahasa Indonesia:

STOPWORDS = sastrawi_stopwords $\cup$ _extra_stopwords

_extra_stopwords berisi 80+ kata informal media sosial yang tidak ada di kamus Sastrawi, termasuk singkatan (yg, dgn, bgt), kata gaul (gw, lo, aja), ekspresi tawa (haha, wkwk), kata ganti, dan kata generik yang tidak bermakna untuk analisis sentimen.

SLANG_DICT (kamus normalisasi kata slang → bentuk baku):

Kata Slang	Bentuk Baku	Kata Slang	Bentuk Baku
gak/ga/nggak	tidak	gw/gue	saya
udh/udah	sudah	lo/lu/elo	kamu
bgt/bngt	banget	klo/klu	kalau
emg/emang	memang	cewe/cewek	perempuan
bs/bsa	bisa	cowo/cowok	laki-laki
org	orang	sexual/sexi	seksual

7.3 Fungsi Preprocessing Per Tahap

case_folding(text: str) → str

Tahap 1 — Case Folding: Mengubah semua karakter teks ke huruf kecil (lowercase) menggunakan `text.lower().strip()`. Memastikan konsistensi representasi: "Pelecehan" == "pelecehan" == "PELECEHAN".

Parameter:

text — String teks asli

Return:

String dalam huruf kecil, tanpa leading/trailing whitespace

cleansing(text: str) → str

Tahap 2 — Cleansing: Membersihkan teks dari 8 jenis noise secara berurutan menggunakan regex: (1) URL → hapus, (2) Mention @user → hapus, (3) Hashtag #topik → hapus, (4) Emoji & non-ASCII → hapus via encode/decode ASCII, (5) Angka → hapus, (6) Tanda baca & simbol → ganti spasi, (7) Underscore → ganti spasi, (8) Spasi berlebih → normalisasi satu spasi.

Parameter:

text — Teks setelah case_folding

Return:

Teks bersih: hanya huruf alfabet dan spasi tunggal

normalize_slang(text: str) → str

Tahap 3 — Normalisasi Slang: Mengganti kata slang/alay ke bentuk bakunya menggunakan `SLANG_DICT`. Proses: split teks → cek setiap token di kamus → ganti jika ada → join kembali. Token yang tidak ada di kamus dikembalikan apa adanya.

Parameter:

text — Teks setelah cleansing

Return:

Teks dengan kata-kata slang sudah diganti ke bentuk baku

tokenizing(text: str) → list

Tahap 4 — Tokenizing: Memecah teks menjadi daftar token menggunakan `NLTK word_tokenize()` dengan mode Indonesian. Fallback ke `text.split()` jika NLTK gagal.

Parameter:

text — Teks setelah normalisasi slang

Return:

`list[str]` — daftar token kata individual

stopword_removal(tokens: list) → list

Tahap 5 — Stopword Removal: Menyaring token: hapus kata yang ada di `STOPWORDS` atau panjangnya ≤ 2 karakter. Menggunakan operasi lookup $O(1)$ pada set `STOPWORDS`.

Parameter:

tokens — Daftar token dari tokenizing

Return:

`list[str]` — token yang bermakna saja

```
stemming(tokens: list) → list
```

Tahap 6 — Stemming: Mengubah token ke kata dasarnya menggunakan Sastrawi Stemmer (algoritma Nazief-Adriani). Contoh: "melaporkan" → "lapor", "pelecehan" → "leceh", "menggoda" → "goda". Jika Sastrawi tidak tersedia, token dikembalikan tanpa perubahan (identity transform).

Parameter:

tokens — Daftar token setelah stopword removal

Return:

list[str] — token dalam bentuk kata dasar

7.4 Pipeline Utama

```
preprocess(text: str) → str
```

Menjalankan keenam tahap preprocessing secara berurutan dalam satu panggilan. Urutan: case_folding → cleansing → normalize_slang → tokenizing → stopword_removal → stemming. Ditambah filter akhir untuk menghapus token ≤ 1 karakter. Contoh: "@user2024 Ini pelecehan seksual banget!!!" → "leceh seksual"

Parameter:

text — Teks komentar asli dari platform X

Return:

String teks bersih siap TF-IDF. String kosong "" jika input tidak valid atau semua token terhapus

```
preprocess_batch(texts: list) → list
```

Menjalankan preprocess() untuk setiap elemen dalam daftar input secara berurutan. Digunakan saat upload dataset (preprocessing ratusan/ribuan teks sekaligus) dan saat pipeline training membutuhkan teks segar.

Parameter:

texts — list[str] — daftar teks komentar asli

Return:

list[str] — hasil preprocessing, panjang sama dengan input

Catatan: Proses ini bisa memakan beberapa menit untuk dataset besar karena stemming Sastrawi memproses per token

■ 8. File: pipeline.py — Machine Learning Pipeline

File **pipeline.py** mengimplementasikan pipeline machine learning end-to-end dari data mentah hingga evaluasi model. Mengintegrasikan semua komponen: preprocessing, TF-IDF, split data, SMOTE, training ID3 Modifikasi, dan evaluasi metrik.

Alur pipeline lengkap:

```
Data Mentah (texts, labels)
↓ preprocess_batch() [atau gunakan preprocessed_text dari DB]
Teks Bersih
↓ TfidfVectorizer(max_features=300, ngram_range=(1,2), min_df=2, sublinear_tf=True)
Matriks Fitur TF-IDF
↓ train_test_split(stratify=y, random_state=42)
Train Set + Test Set
↓ SMOTE(k_neighbors=min(5,n_minority-1)) [jika use_smote=True]
Balanced Train Set
↓ ID3ModifiedClassifier(max_depth=max_depth).fit()
Model Terlatih
↓ model.predict(X_test)
Prediksi → Evaluasi (accuracy, precision, recall, F1, CM)
```

8.1 Fungsi run_experiment()

```
run_experiment(texts, labels, train_ratio=0.8, max_depth=3, use_smote=True,
max_features=300, preprocessed_texts=None) → dict
```

Fungsi inti pipeline yang menjalankan satu siklus eksperimen lengkap. Menggabungkan semua tahap dari preprocessing hingga evaluasi. Optimasi: jika `preprocessed_texts` sudah disediakan dari database, tahap preprocessing dilewati. SMOTE diterapkan HANYA pada training set; testing set dibiarkan asli untuk evaluasi realistis. Waktu training dan prediksi diukur terpisah menggunakan `time.perf_counter()`.

Parameter:

texts — list[str] — teks komentar asli
labels — list[str] — label kelas ("positif"/"negatif")
train_ratio — float — proporsi data training (contoh: 0.8 untuk split 80:20)
max_depth — int — kedalaman maksimum pohon keputusan
use_smote — bool — aktifkan SMOTE oversampling
max_features — int — jumlah fitur TF-IDF maksimum
preprocessed_texts — list[str]|None — teks preprocessed dari DB (opsional)

Return:

dict komprehensif: `preprocessed`, `vectorizer`, `model`, `le`, `class_names`, `X_train/test`, `y_train/test/pred`, `train_count`, `test_count`, `accuracy`, `precision`, `recall`, `f1`, `tp/tn/fp/fn`, `confusion_matrix`, `report`, `train_ratio`, `test_ratio`, `max_depth`, `use_smote`, `execution_time`, `train_time`, `predict_time`

8.2 Fungsi run_depth_sweep()

```
run_depth_sweep(texts, labels, train_ratio=0.8, depths=None, use_smote=True,
max_features=300, preprocessed_texts=None) → list
```

Menjalankan eksperimen untuk berbagai nilai max_depth (default: 1–20) untuk menghasilkan data grafik "Akurasi vs. Max Depth". Optimasi efisiensi: preprocessing, TF-IDF, split, dan SMOTE dijalankan SEKALI saja di awal; loop hanya mengulang tahap training dan prediksi.

Parameter:

depths — list[int]|None — nilai depth yang diuji, default range(1,21)

Return:

list[dict] — setiap dict berisi {max_depth, accuracy, precision, recall, f1, execution_time}

8.3 Manajemen Model

```
save_model(experiment_id, vectorizer, model, le) → str
```

Menyimpan tiga komponen model (TF-IDF vectorizer, ID3 classifier, LabelEncoder) ke satu file pickle di direktori models/. Nama file: model_{experiment_id}.pkl. File lama akan ditimpa.

Return:

str — path absolut file .pkl yang disimpan

```
load_model(experiment_id) → (vectorizer, model, le)
```

Memuat komponen model dari file pickle model_{experiment_id}.pkl. Mengembalikan (None, None, None) jika file tidak ditemukan.

Return:

tuple (TfidfVectorizer, ID3ModifiedClassifier, LabelEncoder) atau (None, None, None)

8.4 Fungsi predict_single()

```
predict_single(text: str, experiment_id: int) → dict
```

Entry point prediksi untuk satu teks baru. Menggabungkan: load model → preprocess → vectorize → predict → decode label. Digunakan oleh route /classify dan API /api/classify.

Parameter:

text — str — teks komentar yang akan diklasifikasikan (teks mentah)

experiment_id — int — ID eksperimen (model) yang akan digunakan

Return:

dict sukses: {label, confidence, preprocessed, raw_pred} | dict error: {error: pesan}

■ 9. File: id3_modified.py — Algoritma ID3 Modifikasi

File **id3_modified.py** adalah implementasi custom dari algoritma ID3 Modifikasi berdasarkan jurnal Asrianda et al. (2025). Inti modifikasi adalah penggantian formula Shannon Entropy (menggunakan $\log_2$) dengan formula entropy berbasis logaritma natural ($\ln$) yang dinormalisasi ke basis-2.

9.1 Formula Matematika

Referensi jurnal:

Asrianda, A., Mawengkang, H., Sihombing, P., Nasution, M. K. M. (2025). "Optimization of Marketing Campaigns Using a Modified ID3 Decision Tree Algorithm." *Eastern-European Journal of Enterprise Technologies*, 2(13), 58–70.

Formula	Persamaan	Keterangan
Entropy Dataset	$Ent(D) = (1/\ln 2) \times [\ln(p+n) - (p \cdot \ln(p) + n \cdot \ln(n)) / (p+n)]$	p=jumlah positif, n=jumlah negatif, $\ln$ =log natural
Entropy Atribut	$EntA(D) = \sum (D_j / D) \times (1/\ln 2) \times [\ln(D_j) - (p_j/ D_j) \cdot \ln(p_j) - (n_j/ D_j) \cdot \ln(n_j)]$	D_j =subset ke-j, p_j =positif di D_j , n_j =negatif di D_j
Information Gain	$Gain(A) = Ent(D) - EntA(D)$	Semakin tinggi Gain, semakin baik atribut A memisahkan kelas

9.2 Fungsi Entropy dan Information Gain

modified_entropy(p: int, n: int) → float

Menghitung entropy modifikasi menggunakan formula $Ent(D)$ dari Asrianda et al. (2025). Mengembalikan 0.0 jika total data nol atau dataset murni satu kelas. Nilai 0.0 = dataset murni (tidak ada ketidakpastian), nilai ~1.0 = distribusi seimbang (ketidakpastian maksimum).

Parameter:

p — jumlah instance kelas positif

n — jumlah instance kelas negatif

Return:

float — nilai entropy dalam satuan bits (0.0 hingga 1.0)

modified_conditional_entropy(y_parent, x_col, y) → float

Menghitung Conditional Entropy ($EntA(D)$) untuk fitur binary (hasil threshold). Iterasi setiap nilai unik di x_col , hitung entropy subset menggunakan formula modifikasi, lalu hitung rata-rata berbobot berdasarkan proporsi ukuran subset.

Parameter:

x_col — np.ndarray — array 1D nilai fitur binary (0 atau 1)

y — np.ndarray — label kelas aktual (0 atau 1)

Return:

float — nilai conditional entropy dalam bits

information_gain(y, X_col) → float

Menghitung Information Gain: $\text{Gain}(A) = \text{Ent}(D) - \text{Ent}(A|D)$. Nilai tinggi berarti fitur sangat informatif untuk memisahkan kelas.

Parameter:

y — np.ndarray — label kelas aktual

X_col — np.ndarray — nilai fitur binary hasil threshold

Return:

float — Information Gain dalam bits

9.3 Kelas ID3ModifiedNode

ID3ModifiedNode adalah struktur data untuk satu node dalam pohon keputusan. Setiap node menyimpan: **feature_index** (fitur splitting), **threshold** (nilai pemisah), **left/right** (node anak), **is_leaf** (apakah daun), **prediction** (prediksi kelas jika leaf), dan **depth** (kedalaman dalam pohon).

9.4 Kelas ID3ModifiedClassifier

ID3ModifiedClassifier adalah classifier utama dengan antarmuka kompatibel sklearn.

Method	Signature	Keterangan
<code>__init__()</code>	<code>max_depth=3, min_samples_split=2</code>	Inisialisasi classifier dengan parameter kedalaman dan minimum sampel split
<code>fit()</code>	<code>X, y → self</code>	Training: bangun pohon keputusan rekursif, hitung <code>feature_importances_</code> dari akumulasi Information Gain
<code>_build_tree()</code>	<code>X, y, depth → ID3ModifiedNode</code>	Fungsi rekursif internal: cari split terbaik via Information Gain, buat node internal atau leaf
<code>_predict_one()</code>	<code>x, node → int</code>	Traversal pohon rekursif untuk satu sampel: kiri jika $\leq$ threshold, kanan jika $>$
<code>predict()</code>	<code>X → np.ndarray</code>	Batch prediction: panggil <code>_predict_one()</code> untuk setiap baris X
<code>predict_proba()</code>	<code>X → np.ndarray</code>	Estimasi probabilitas (simplified): kelas diprediksi = 0.85, lainnya = 0.15

Strategi binary split pada data TF-IDF numerik:

Karena TF-IDF menghasilkan nilai numerik kontinu, **ID3ModifiedClassifier** menggunakan binary split: untuk setiap fitur, semua titik tengah (midpoint) antara nilai unik berurutan dicoba sebagai kandidat threshold. Threshold dengan Information Gain tertinggi dipilih. Sampel dengan nilai $\leq$ threshold ke kiri, $>$ threshold ke kanan.

■ ■ 10. Template HTML (Folder templates/)

Folder **templates/** berisi 14 file template HTML yang menggunakan Jinja2 templating engine. Semua template mewarisi dari base.html menggunakan blok extends/block.

File Template	Halaman	Akses	Keterangan
base.html	Layout Dasar	Semua	Template induk: navbar, footer, struktur HTML global, block konten
index.html	Beranda	Publik	Tampilkan info model terbaik (akurasi, max_depth), statistik prediksi, CTA ke /classify
classify.html	Klasifikasi	Publik	Form input teks, tampilkan label prediksi (positif/negatif), confidence, teks preprocessed
about.html	Tentang	Publik	Informasi sistem: penjelasan ID3 Modifikasi, formula, metodologi, referensi jurnal
login.html	Login Admin	Publik	Form login admin: input username & password, tampilkan error jika gagal
dashboard.html	Dashboard	Admin	Kartu statistik ringkasan: total dataset/eksperimen/prediksi, tabel dataset & eksperimen terbaru
datasets.html	List Dataset	Admin	Tabel daftar dataset: nama file, jumlah data, distribusi kelas, tombol detail & hapus
upload_dataset.html	Upload Dataset	Admin	Form upload file CSV, field catatan opsional, panduan format kolom yang diterima
dataset_detail.html	Detail Dataset	Admin	Info metadata dataset, tabel preview data 20 baris/halaman, list eksperimen terkait
run_experiment.html	Konfigurasi Training	Admin	Form pilih dataset, checkbox proporsi (80:20/70:30), input max_depth, toggle SMOTE & depth sweep
experiment_progress.html	Progress Training	Admin	Progress bar real-time via JavaScript polling ke /api/task-status/
experiments.html	List Eksperimen	Admin	Tabel semua eksperimen selesai: akurasi, F1, waktu, badge "Terbaik", link detail
experiment_detail.html	Detail Eksperimen	Admin	Metrik lengkap, confusion matrix, tabel per kelas, grafik akurasi vs depth (Chart.js)
statistics.html	Statistik	Admin	Grafik perbandingan eksperimen, distribusi label prediksi, tren akurasi, prediksi terbaru
predictions.html	Riwayat Prediksi	Admin	Tabel 100 prediksi terbaru: teks input, hasil, confidence, waktu, statistik distribusi

■ 11. File Models & Uploads

Folder models/

Folder **models/** menyimpan file model terlatih dalam format pickle (.pkl). Setiap eksperimen yang berhasil diselesaikan menghasilkan satu file model.

Aspek	Detail
Penamaan file	model_{experiment_id}.pkl
Isi file pickle	Dictionary berisi tiga key: "vectorizer" (TfidfVectorizer), "model" (ID3ModifiedClassifier), "le" (LabelEncoder)
Cara simpan	pipeline.save_model(experiment_id, vectorizer, model, le)
Cara muat	pipeline.load_model(experiment_id) → (vectorizer, model, le)
Ukuran file	Bervariasi tergantung max_features dan depth (umumnya beberapa KB hingga MB)
File tersedia	model_1.pkl, model_2.pkl (dua model dari eksperimen sebelumnya)

Folder uploads/

Folder **uploads/** menyimpan file CSV yang diunggah admin secara sementara. File CSV dibaca saat proses upload untuk validasi dan preprocessing, kemudian datanya disimpan ke database Supabase.

File tersedia: **data_skripsi.csv** — dataset komentar platform X tentang pelecehan seksual verbal.

■ 12. Alur Kerja Sistem (End-to-End)

12.1 Alur Admin: Training Model Baru

Langkah 1: Upload Dataset

- Admin login di `/admin/login` dengan username & password
- Buka `/admin/datasets/upload`, pilih file CSV
- Sistem validasi format, baca CSV (multi-encoding fallback)
- Preprocessing batch semua teks (case folding → cleansing → normalisasi → tokenizing → stopwords → stemming)
- Insert metadata ke tabel `datasets` dan data ke `raw_data` (batch 500)

Langkah 2: Konfigurasi Eksperimen

- Buka `/admin/experiments/run`
- Pilih dataset, centang proporsi (80:20 dan/atau 70:30)
- Set `max_depth` (rekomenasi: 3–6 berdasarkan jurnal)
- Toggle SMOTE dan/atau depth sweep jika diperlukan
- Submit form → sistem buat `task_id` unik

Langkah 3: Training Background

- Background thread dimulai (`daemon=True`)
- Muat semua data dari DB, gunakan `preprocessed_text` jika tersedia
- TF-IDF → split stratified → SMOTE → training ID3 Modifikasi
- Simpan hasil ke tabel `experiments` + `experiment_details`
- Simpan model ke `models/model_{id}.pkl`
- Opsional: jalankan depth sweep 1-20 dan simpan ke `depth_results`

Langkah 4: Analisis Hasil

- Buka `/admin/experiments` untuk lihat daftar eksperimen selesai
- Klik eksperimen untuk detail: akurasi, precision, recall, F1, confusion matrix
- Lihat grafik akurasi vs `max_depth` jika depth sweep dijalankan
- Bandingkan antar proporsi (80:20 vs 70:30)
- Klik "Set Model Terbaik" pada eksperimen dengan performa terbaik

12.2 Alur Pengguna: Klasifikasi Teks

1. Pengguna buka `/classify`
2. Input teks komentar (minimal 5 karakter)
3. Sistem memanggil `get_best_experiment()` untuk cari model aktif
4. `predict_single(text, exp_id) → preprocess → vectorize → model.predict()`
5. Tampilkan: label (positif/negatif), confidence (%), teks preprocessed
6. Hasil disimpan otomatis ke tabel `predictions` untuk analisis admin

■ 13. API Reference

VerbaClassify menyediakan beberapa REST API endpoint yang dapat digunakan untuk integrasi dengan sistem eksternal atau testing programmatic.

POST /api/classify

Klasifikasi teks komentar via JSON. Tidak memerlukan autentikasi.

Request body:

```
{ "text": "teks komentar yang akan diklasifikasikan" }
```

Response sukses (200):

```
{
  "label" : "positif" | "negatif",
  "confidence" : 0.85,
  "preprocessed": "teks setelah preprocessing"
}
```

Response error:

Status	Kondisi
400	Teks kosong atau prediksi gagal (teks kosong setelah preprocessing)
500	Exception internal server
503	Model belum tersedia (belum ada eksperimen selesai)

GET /api/stats

Statistik ringkasan sistem. Tidak memerlukan autentikasi.

Response (200):

```
{
  "total_datasets" : 3,
  "total_experiments" : 5,
  "total_predictions" : 120,
  "best_accuracy" : 0.84,
  "best_experiment" : { ...record eksperimen... },
  "pred_positif" : 75,
  "pred_negatif" : 45
}
```

GET /api/task-status/

Status background task training. Memerlukan autentikasi admin.

```
{ "status": "running"|"done"|"error"|"not_found", "progress": 65, "message": "Training proporsi 80:20..." }
```

GET /api/experiments//depth-results

Data hasil sweep max_depth untuk grafik Chart.js. Tidak memerlukan autentikasi.

```
[  
{"max_depth": 1, "accuracy": 0.82, "precision": 0.81, "recall": 0.82, "f1": 0.81,  
"execution_time": 0.12},  
{"max_depth": 2, "accuracy": 0.84, ...},  
...  
{"max_depth": 20, "accuracy": 0.79, ...}  
]
```

■ ■ 14. Panduan Setup & Instalasi

1. Kloning / Ekstrak Proyek

- Ekstrak file ZIP VerbaClassify.zip ke direktori pilihan
- Masuk ke direktori: `cd VerbaClassify/skripsi_app/`

2. Buat Virtual Environment (opsional tapi direkomendasikan)

```
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate # Windows
```

3. Install Dependensi

```
pip install -r requirements.txt
python -m nltk.downloader punkt punkt_tab stopwords # Download NLTK data
```

4. Setup Database Supabase

- Buka Supabase Dashboard → <https://supabase.com/dashboard>
- Buat project baru atau gunakan yang sudah ada
- Buka SQL Editor, paste isi file `schema.sql`, klik Run
- Copy URL dan API key project

5. Konfigurasi .env

- Salin `.env.example` menjadi `.env` (atau edit langsung file `.env`)
- Isi `SUPABASE_URL` dan `SUPABASE_KEY` dari dashboard Supabase
- Ubah `ADMIN_PASSWORD` menjadi password yang kuat

6. Jalankan Aplikasi

```
python app.py
```

- Buka browser ke `http://localhost:5000`
- Login admin di `http://localhost:5000/admin/login`

■ 15. Dependensi (requirements.txt)

Package	Versi Min	Kegunaan dalam Sistem
flask	>=3.0.0	Web framework utama: routing, templating Jinja2, session management
supabase	>=2.0.0	Client library Python untuk koneksi ke Supabase (PostgreSQL)
python-dotenv	>=1.0.0	Membaca variabel konfigurasi dari file .env
pandas	>=2.0.0	Parsing dan validasi file CSV saat upload dataset
numpy	>=1.24.0	Operasi array: np.unique, np.bincount, np.zeros, dll. di id3_modified.py
scikit-learn	>=1.3.0	TfidfVectorizer, train_test_split, LabelEncoder, metrik evaluasi
imbalanced-learn	>=0.11.0	SMOTE (Synthetic Minority Over-sampling Technique) untuk balancing kelas
nltk	>=3.8.0	word_tokenize() untuk tokenisasi teks Bahasa Indonesia
PySastrawi	>=1.2.0	Stemmer Nazief-Adriani dan daftar stopwords Bahasa Indonesia
Werkzeug	>=3.0.0	secure_filename() untuk sanitasi nama file upload
gunicorn	>=21.0.0	Production WSGI server untuk deployment di server Linux

Dokumentasi VerbaClassify

Dokumen ini mencakup dokumentasi teknis lengkap seluruh komponen sistem VerbaClassify — dari struktur file, fungsi-fungsi Python, schema database, hingga panduan instalasi dan API reference.

Rizka Mardiah Putri Buyung Lubis | NIM: 220170183
Teknik Informatika — Universitas Malikussaleh | 2026

Dokumen dibuat: 10 June 2026, 05:42
